

DART XML 검색 청크 생성 규칙

항목	내용
문서 버전	v1.0
작성일	2026-08-19
문서 상태	초기 구현 규칙 확정본
적용 대상	파싱·적재가 완료된 대회 제공 DART_XML 문서
선행 문서	05_dart_xml_parsing_and_validation.md, 06_dart_xml_parsing_persistence.md
입력 테이블	disclosure_sections, disclosure_content_blocks
예정 출력	disclosure_chunks와 원본 블록 연결 정보
생성 규칙 버전	dart-xml-chunk-v1

1. 문서 목적

이 문서는 파싱된 DART XML의 Section과 ContentBlock을 검색 가능한 텍스트 청크로 변환하는 규칙을 정의한다.

검색 청크의 목적은 다음과 같다.

- 사용자 질문과 관련된 원문 구간을 빠르게 찾는다.
- HyperCLOVA X에 전체 공시가 아니라 필요한 문맥만 전달한다.
- 검색 결과에서 원본 공시·Section·Block·원문 행으로 역추적한다.
- 서술형 질문과 아직 Fact로 정의하지 않은 질문에 대응한다.
- Fact 후보가 존재할 가능성이 높은 원문 범위를 좁힌다.

검색 청크는 원문이나 Fact를 대체하지 않는다.

원문 ContentBlock

└─ 검색 청크: 관련 내용을 찾기 위한 텍스트

└─ Fact: 계산·비교 가능한 정규화 값

금액·날짜·비율 계산은 청크 문자열이 아니라 검증된 Fact를 사용한다. Fact의 최종 근거도 검색 청크만 가리키지 않고 실제 원본 ContentBlock과 원문 위치를 가리켜야 한다.

2. 적용 범위와 제외 범위

2.1 이번 규칙의 적용 범위

- DART XML의 Section 경로 생성
- preamble 처리
- HEADING·PARAGRAPH를 이용한 TEXT 청크 생성
- TABLE JSONB를 이용한 TABLE 청크 생성

- rowspan-colspan을 고려한 검색 텍스트 생성
- 중첩 표 처리
- 의미 있는 이미지 캡션 처리
- PAGE_BREAK 처리
- 원본 Block과 원문 행 연결
- 청크 생성 버전과 재생성 원칙
- 소규모·전체 배치 검증 기준

2.2 이번 규칙의 제외 범위

- disclosure_chunks의 최종 Flyway 스키마
- PostgreSQL 전문 검색 인덱스의 최종 방식
- 임베딩 모델과 Vector DB
- HTML-PDF 청크 규칙
- 공시 유형별 Fact 추출 규칙
- QueryPlan과 검색 API 구현
- LLM 답변 프롬프트

이 문서에서 출력 계약은 논리 구조를 정의한다. 실제 컬럼과 연관 테이블은 소규모 생성 결과를 검증한 뒤 확정한다.

3. 현재 입력 데이터의 특징

DART XML 3,147개 파싱·적재 결과는 다음과 같다.

구조	개수
문서	3,147
Section	86,343
전체 최상위 ContentBlock	2,587,948
HEADING	56,619
PARAGRAPH	849,925
PAGE_BREAK	117,870
최상위 TABLE 블록	1,559,230
최상위 IMAGE 블록	4,304
중첩 TABLE	18,165
최상위·중첩 표 전체	1,577,395
표 행	8,054,877
표 셀	23,192,778

현재 구조에서 특히 고려해야 할 점은 다음과 같다.

1. TABLE이 최상위 ContentBlock의 약 60%를 차지한다.
2. 표 행 하나마다 청크를 만들면 청크 수가 지나치게 많아질 수 있다.

3. 많은 표는 행 수가 적지만 일부 표는 매우 크다.
4. 중첩 표는 별도 ContentBlock이 아니라 부모 TABLE JSONB의 셀 안에 저장된다.
5. 표 내부의 P는 별도 PARAGRAPH가 아니라 셀 텍스트의 줄바꿈으로 보존된다.
6. 표 셀 안의 이미지는 TABLE JSONB 안에 저장된다.
7. 일반 본문의 이미지만 독립 IMAGE 블록으로 저장된다.
8. Section은 1~4단계 계층과 원문 범위를 보존한다.
9. Section 이전의 preamble 블록은 `section_id=NULL`이다.
10. Section과 Block의 `sequence_no`를 합치면 문서 전체 원문 순서를 복원할 수 있다.

따라서 모든 내용을 일정 글자 수로 자르는 방식이 아니라 Section·Block·문장·표 행을 우선하는 구조 기반 청킹을 사용한다.

4. 핵심 설계 원칙

4.1 구조 우선, 크기 후순위

분할 경계의 우선순위는 다음과 같다.

```

문서 경계
→ Section 경계
→ HEADING·PARAGRAPH·TABLE 경계
→ 문장 또는 표 행 경계
→ 줄바꿈
→ 마지막 수단으로 글자 수 분할
  
```

글자 수는 의미 구조를 무시하고 자르는 절대 기준이 아니라 합치기·나누기 판단을 돕는 제한값이다.

4.2 고정 크기가 아닌 목표 범위

모든 청크를 같은 길이로 만들지 않는다.

```

짧지만 의미가 완결된 Section → 짧은 청크 허용
인접한 짧은 문단 → 목표 범위까지 결합
긴 문단 → 문장 경계로 분리
큰 표 → 행 묶음으로 분리
행 하나가 큰 표 → 행의 의미 보존을 우선
  
```

4.3 원문 보존과 파생 데이터 분리

- ContentBlock과 TABLE JSONB는 변경하지 않는다.
- 청크는 검색용으로 재구성한 파생 데이터다.
- rowspan 확장, 머리글 반복 등은 검색 텍스트에만 적용한다.
- 청크를 삭제하거나 재생성해도 파싱 원문은 영향을 받지 않는다.

4.4 검색 정확도와 근거 정확도를 함께 보장

검색 텍스트는 사람이 읽고 검색하기 좋아야 하지만, 결과가 실제 원문 위치로 연결되지 않으면 안 된다.

각 청크는 최소한 다음을 추적할 수 있어야 한다.

- 원문 문서
- Section과 전체 Section 경로
- 사용한 ContentBlock 목록과 순서
- 원문 시작·종료 행
- 표라면 원본 TABLE, 행, 셀 또는 JSON 위치
- 청크 생성 규칙 버전

4.5 저장 중복을 최소화하고 검색 시 문맥을 확장

일반 본문을 일정 비율로 중복 저장하는 overlap은 v1에서 사용하지 않는다.

검색 시 선택된 청크의 앞뒤 청크를 같은 Section 안에서 추가 조회해 문맥을 확장한다.

검색 적중: 청크 20
LLM 전달 후보: 청크 19 + 청크 20 + 청크 21

단, Section 경로와 표 머리글은 각 청크의 의미를 독립적으로 이해하기 위해 반복할 수 있다.

5. 청크 유형

초기 버전은 다음 세 유형을 사용한다.

청크 유형	입력	목적
TEXT	HEADING·PARAGRAPH	일반 본문과 서술형 검색
TABLE	TABLE JSONB	표 레이블·행·값 검색
IMAGE_CAPTION	IMAGE 또는 셀 이미지 캡션	이미지 존재와 설명 검색 보조

다음 항목은 독립 청크를 만들지 않는다.

입력	처리
PAGE_BREAK	소프트 분할 경계로만 사용
제목만 있는 HEADING	다음 본문 청크의 문맥으로 사용
파일명만 있고 의미 없는 이미지	검색 청크 생성 안 함
내용이 없는 표·행·셀	텍스트 청크 생성 안 함

TEXT와 TABLE은 하나의 청크에 섞지 않는다. 여러 개의 작은 TABLE 조각은 조건을 만족할 때 같은 TABLE 청크로 묶을 수 있다.

6. 출력 청크의 논리 계약

청크 생성기의 인메모리 결과는 최소한 다음 정보를 제공해야 한다.

```
GeneratedDisclosureChunk
├─ documentId
├─ sectionId           // preamble이면 NULL
├─ sectionPath
├─ chunkType
├─ chunkSequenceNo
├─ bodyText
├─ searchText
├─ sourceBlockIds
├─ sourceLineStart
├─ sourceLineEnd
├─ firstSourceSequenceNo
├─ lastSourceSequenceNo
├─ characterCount
├─ generatorName
└─ generatorVersion
```

TABLE 청크는 추가로 다음 출처 정보를 표현할 수 있어야 한다.

```
TableSourceLocation
├─ contentBlockId
├─ tableOrder
├─ nestingPath
├─ rowIndexStart
├─ rowIndexEnd
└─ cellIndexes 또는 JSON 경로
```

bodyText는 원문 내용을 검색 가능한 형태로 직렬화한 본문이다. **searchText**는 공시명·Section 경로·소제목 등 검색 문맥을 포함할 수 있다.

같은 회사명·접수번호를 모든 **bodyText**에 반복할 필요는 없다. 기업·기간·공시 유형은 DB 컬럼으로 먼저 필터링하고, Section 경로와 소제목은 본문 검색 품질을 위해 **searchText**에 포함한다.

7. 문서와 Section 처리 규칙

7.1 문서 경계

서로 다른 **DisclosureDocument**의 블록은 어떤 경우에도 같은 청크로 합치지 않는다.

7.2 Section 경계

서로 다른 Section의 본문은 기본적으로 합치지 않는다.

Section이 짧더라도 다음 Section과 합쳐 최소 길이를 맞추지 않는다. 최소 길이는 강제 조건이 아니라 품질 검사 지표다.

7.3 전체 Section 경로

부모 Section을 따라 전체 경로를 만든다.

```

II. 사업의 내용
└ 1. 사업의 개요
  └ 가. 주요 제품

```

검색 문맥:

[II. 사업의 내용 > 1. 사업의 개요 > 가. 주요 제품]

제목이 없는 Section은 빈 제목을 경로에 추가하지 않는다. 자식 Section은 제목이 있는 가장 가까운 조상 경로를 유지한다.

7.4 preamble

`section_id=NULL`인 첫 Section 이전 블록은 삭제하지 않는다.

다음 가상 경로를 부여하고 일반 규칙을 적용한다.

[문서 서두]

7.5 원문 순서

Section과 ContentBlock의 `sequence_no`를 기준으로 문서 원문 순서를 유지한다.

청크 `chunkSequenceNo`는 문서 안에서 1부터 연속 증가한다. 이는 원문 `sequence_no`와 별개의 검색 청크 순서다.

8. TEXT 청크 규칙

8.1 입력 범위

TEXT 청크는 같은 문서, 같은 Section 안의 HEADING과 PARAGRAPH를 사용한다.

TABLE을 만나면 누적 중인 TEXT 청크를 먼저 종료한다. TABLE 이후의 문단은 새로운 TEXT 청크에서 시작한다.

8.2 HEADING 처리

HEADING은 기본적으로 단독 청크로 만들지 않고 뒤에 오는 PARAGRAPH의 문맥으로 사용한다.

```

HEADING: 주요 제품 및 서비스
PARAGRAPH: 당사는 반도체 장비를 제조하고 있으며...

```

결과:

[Section 경로]
소제목: 주요 제품 및 서비스

당사는 반도체 장비를 제조하고 있으며...

새 HEADING을 만나면 다음과 같이 처리한다.

1. 누적 TEXT에 본문이 있으면 현재 청크를 종료한다.
2. 새 HEADING을 이후 문단의 문맥으로 저장한다.
3. 연속 HEADING은 계층 문맥처럼 순서대로 보존한다.
4. Section 끝까지 본문이 없으면 제목만으로 독립 청크를 만들지 않는다.

8.3 짧은 문단 결합

같은 Section과 같은 HEADING 문맥에 있는 인접 PARAGRAPH를 목표 범위까지 합친다.

문단 1: 220자
 문단 2: 310자
 문단 3: 280자
 → 약 810자의 TEXT 청크

문단 사이에는 빈 줄 하나를 넣어 원래 문단 경계를 식별할 수 있게 한다.

8.4 긴 문단 분리

PARAGRAPH 하나가 최대 크기를 넘으면 다음 순서로 분리한다.

문장 종결 경계
 → 보존된 줄바꿈
 → 구두점 경계
 → 마지막 수단으로 글자 수 경계

긴 문단을 나눈 모든 청크에 같은 Section 경로와 HEADING 문맥을 붙인다.

문장 분리는 숫자의 소수점, 날짜, 약어를 문장 종결로 잘못 인식하지 않도록 테스트해야 한다.

8.5 TEXT 크기 설정

`dart-xml-chunk-v1` 초기 설정값은 다음과 같다.

설정	값	의미
목표 하한	700자	가능하면 이 길이 이상까지 인접 문단 결합
목표 상한	1,000자	새 문단 추가 여부를 판단하는 목표
일반 최대	1,400자	문단 경계를 지키면서 허용하는 최대
절대 최대	2,000자	긴 문단 분리가 반드시 필요한 기준

규칙:

- 700자 미만이라고 다른 Section과 합치지 않는다.
- 문단 하나가 1,000자를 조금 넘더라도 1,400자 이하면 그대로 유지할 수 있다.
- 문단 하나가 1,400자를 넘으면 문장 단위 분리를 시도한다.
- 생성 결과는 원칙적으로 2,000자를 넘지 않는다.
- Section 경로·HEADING을 포함한 **searchText** 길이와 실제 **bodyText** 길이를 모두 기록한다.

이 수치는 데이터 불변 규칙이 아니라 **v1** 운영 설정이다. 검증 결과에 따라 값이 바뀌면 생성 버전을 함께 변경한다.

9. TABLE 청크 규칙

9.1 기본 원칙

- TABLE은 TEXT와 별도 청크로 생성한다.
- 표 전체 JSON을 문자열로 그대로 저장하지 않는다.
- 표의 행과 셀을 사람이 읽을 수 있는 검색 텍스트로 직렬화한다.
- 표 제목·Section 경로·머리글·행 레이블·단위를 가능한 한 보존한다.
- 행 하나를 서로 다른 청크로 나누지 않는 것을 우선한다.
- Fact 추출은 검색용 문자열이 아니라 원본 TABLE JSONB를 기준으로 한다.

9.2 논리적 표 그리드

현재 TABLE JSONB에는 다음 정보가 있다.

- 행 순서와 원문 행
- 셀 순서와 원문 행
- HEADER·DATA 타입
- **rowSpan**
- **colSpan**
- 셀 텍스트와 줄바꿈
- 중첩 표
- 셀 내부 이미지

TABLE 검색 텍스트를 만들기 전에 **rowSpan**과 **colSpan**을 고려한 논리적 그리드를 만든다.

원문 표현:

```
신주의 종류와 수 - 보통주 - 10,000주
                  ↳ 우선주 - 2,000주
```

검색용 표현:

```
신주의 종류와 수 | 보통주 | 10,000주
신주의 종류와 수 | 우선주 | 2,000주
```

원본 셀을 복제하거나 JSONB를 변경하지 않고 검색 텍스트에서만 상위 병합 셀의 문맥을 전달한다.

9.3 머리글 판별

DART XML에서는 머리글처럼 보이는 셀도 **TD**인 경우가 있으므로 **TH**만으로 머리글을 확정하지 않는다.

머리글 처리 우선순위:

1. 명시적인 HEADER 셀이 있으면 사용한다.
2. 연속된 명시적 HEADER 행은 다단 머리글로 보존한다.
3. HEADER가 없다면 원래 셀 순서를 그대로 보존한다.
4. 2열 또는 레이블-값 형태가 명확하면 첫 셀을 레이블처럼 직렬화한다.
5. 의미가 불확실하면 임의의 컬럼명을 생성하지 않는다.

예시:

```
1. 신주의 종류와 수 | 보통주(주) | 16,200,000
```

9.4 행 직렬화

기본 구분자는 **|**를 사용한다.

셀 내부의 보존된 줄바꿈은 의미가 있으므로 무조건 공백으로 없애지 않는다. 한 행의 검색 텍스트에서는 다음 중 하나로 명확히 구분한다.

```
셀 1 | 셀 2의 첫 문장 / 셀 2의 다음 문장 | 셀 3
```

또는 구현 결과의 가독성이 더 좋다면 셀 안의 줄바꿈을 **\n**으로 보존할 수 있다. 선택한 방식은 serializer 테스트에서 고정한다.

빈 셀은 위치가 의미 있는 경우 빈 문자열 자리로 남기고, 행 전체가 비어 있으면 검색 텍스트를 만들지 않는다.

9.5 작은 표

작은 표는 표 하나를 TABLE 조각 하나로 만든다.

인접한 작은 표들은 다음 조건을 모두 만족할 때 하나의 최종 TABLE 청크로 묶을 수 있다.

- 같은 문서다.
- 같은 Section이다.
- 같은 HEADING 문맥이다.
- 사이에 PARAGRAPH 또는 의미 있는 IMAGE가 없다.
- 합친 길이가 TABLE 목표 상한을 넘지 않는다.
- 각 표 사이에 명확한 표 구분자를 넣는다.
- 모든 원본 TABLE Block ID와 행 범위를 보존한다.

예시:

```
[ 표 1 ]
```

```
...
```

[표 2]

...

이 조건부 묶음은 작은 표가 많은 DART XML에서 청크 수가 불필요하게 폭증하는 것을 줄이기 위한 규칙이다.

9.6 큰 표

큰 표는 행을 순서대로 추가해 목표 범위의 TABLE 조각을 만든다.

TABLE 청크 1: 표 문맥 + 머리글 + 1~N행
TABLE 청크 2: 표 문맥 + 머리글 + N+1~M행

분할된 모든 조각에는 다음 내용을 반복한다.

- 전체 Section 경로
- 현재 HEADING 문맥
- 확인 가능한 표 제목 또는 상위 셀 문맥
- 명시적 머리글
- 단위가 별도 행이나 셀에 있다면 단위 문맥

9.7 TABLE 크기 설정

`dart-xml-chunk-v1` 초기 설정값은 다음과 같다.

설정	값	의미
목표 하한	1,000자	행을 계속 결합하는 기준
목표 상한	1,500자	새 행 추가 여부를 판단하는 목표
일반 최대	2,000자	행 경계를 지키면서 허용하는 최대
절대 최대	3,000자	긴 행 내부 분리가 필요한 기준

규칙:

- 다음 행을 추가했을 때 1,500자를 넘으면 현재 조각 종료를 우선한다.
- 행 하나 때문에 2,000자를 조금 넘는 경우 행 의미 보존을 우선할 수 있다.
- 행 하나가 3,000자를 넘으면 행 레이블과 머리글을 반복하면서 긴 셀을 문장·줄바꿈 단위로 나눈다.
- 단순 행 개수보다 직렬화된 글자 수를 우선한다.
- 행 수 상한이 필요하면 메모리·안전용 보조 설정으로 두고 검색 의미를 결정하는 주 기준으로 사용하지 않는다.

9.8 중첩 표

중첩 표는 다음 구조로 저장되어 있다.

```
최상위 TABLE ContentBlock
└─ Row
   └─ Cell
```

```
└─ nestedTables
  └─ ParsedDisclosureTable
```

처리 규칙:

1. 최상위 표부터 재귀적으로 순회한다.
2. 부모 셀의 직접 텍스트는 부모 TABLE 조각에 포함한다.
3. 중첩 표는 독립적인 TABLE 조각 후보로 생성한다.
4. 중첩 표 조각에는 부모 표와 부모 셀의 문맥을 붙인다.
5. 중첩 표의 셀 내용을 부모 표 텍스트에 다시 펼쳐 넣지 않는다.
6. `nestingPath`를 보존해 원본 JSON 위치로 돌아갈 수 있게 한다.

예시:

[연결재무제표 주석 > 특수관계자 거래]
상위 셀 문맥: 특수관계자와의 주요 거래내역

구분		당분기		전분기
매출		...		...

9.9 레이아웃용 부모 표

중첩 표를 담기 위한 외부 레이아웃 표가 존재할 수 있다.

다음 규칙을 적용한다.

- 중첩 표가 없는 일반 표는 정상 직렬화한다.
- 중첩 표가 있는 부모 표는 부모 셀의 직접 텍스트만 직렬화한다.
- 중첩 표 내용은 재귀적으로 별도 조각을 만든다.
- 직접 텍스트가 없고 중첩 표만 담은 부모 행·셀은 별도 검색 텍스트를 만들지 않는다.
- 단지 1행·1셀이라는 이유만으로 내용을 버리지 않는다.

레이아웃 표 판별은 내용 손실 위험이 있으므로 **v1**에서는 보수적으로 처리한다.

10. IMAGE_CAPTION 규칙

10.1 일반 IMAGE 블록

이미지 바이너리는 제공되지 않았으므로 파일명과 캡션 등 XML 메타데이터만 사용한다.

다음 조건을 만족하면 IMAGE_CAPTION 청크를 생성할 수 있다.

- 정규화 후 의미 있는 캡션이 있다.
- 캡션이 . 같은 무의미한 값이 아니다.
- 같은 내용의 본문 텍스트와 명백하게 중복되지 않는다.

예시:

[연결재무제표 주석]

이미지 설명: 연결주석_주1_정정후

원본 파일명: 연결주석_주1_정정후.jpg

10.2 파일명만 있는 이미지

캡션이 없고 1.jpg처럼 의미 없는 파일명만 있으면 독립 검색 청크를 생성하지 않는다.

파일명 자체가 설명적인 경우에도 검색 우선순위를 낮게 두고, 소규모 검증에서 실제 검색 가치가 확인된 뒤 포함한다.

10.3 표 셀 내부 이미지

표 셀 내부 이미지는 TABLE JSONB에 저장되어 있다.

- 의미 있는 캡션은 해당 TABLE 행의 보조 텍스트에 포함할 수 있다.
- 같은 이미지로 독립 IMAGE_CAPTION 청크를 중복 생성하지 않는다.
- 이미지 캡션의 원본 셀과 행 위치를 보존한다.

11. PAGE_BREAK 규칙

PAGE_BREAK는 별도 청크를 생성하지 않는다.

페이지 구분을 만날 때마다 청크를 무조건 종료하면 짧은 청크가 지나치게 많아질 수 있으므로 소프트 경계로만 사용한다.

누적 TEXT가 목표 하한 이상 + PAGE_BREAK

→ 현재 TEXT 청크 종료 가능

누적 TEXT가 목표 하한 미만 + PAGE_BREAK

→ 같은 Section·문맥이면 계속 결합

TABLE 안의 PGBRK는 파서에서 셀 줄바꿈으로 이미 보존됐으므로 TABLE serializer는 보존된 셀 텍스트만 처리한다.

12. 텍스트 정규화 규칙

청크 생성 과정에서 다음 정규화를 적용한다.

- Windows·Unix 줄바꿈을 \n으로 통일한다.
- XML 들여쓰기 때문에 생긴 빈 줄은 제거한다.
- 한 줄 내부의 탭·폼피드·연속 공백은 하나의 공백으로 줄인다.
- 문단 사이 빈 줄과 셀 내부 의미 있는 줄바꿈은 보존한다.
- 금액의 쉼표, 음수 부호, 괄호, %, 통화 단위는 제거하지 않는다.
- 날짜 구분자와 소수점은 유지한다.
- 원문 텍스트를 번역하거나 요약하지 않는다.
- LLM이 만든 문장을 청크 본문으로 저장하지 않는다.

검색 편의를 위한 별도 정규화 문자열이 필요하면 원문형 `bodyText`를 유지한 채 `searchText` 또는 검색 인덱스에서 처리한다.

13. 중복과 overlap 규칙

13.1 저장 overlap

`v1`은 일반 문단의 고정 글자 또는 고정 비율 overlap을 사용하지 않는다.

이유:

- 이미 약 259만 개의 ContentBlock이 있다.
- 표 머리글과 Section 경로만으로도 필요한 문맥이 일부 반복된다.
- overlap은 저장량과 검색 결과 중복을 증가시킨다.
- 근접 청크는 검색 시 동적으로 확장할 수 있다.

13.2 허용되는 반복

다음 반복은 의미 보존을 위해 허용한다.

- Section 전체 경로
- 현재 HEADING 문맥
- 분할된 표의 머리글
- 표 단위와 상위 셀 문맥
- 긴 표 행 분리 시 행 레이블

13.3 금지되는 중복

- 중첩 표 전체를 부모 표와 자식 표 청크에 동시에 저장
- 같은 원본 Block을 생성기 오류로 연속 두 번 추가
- 일반 문단 전체를 앞뒤 청크에 고정 비율로 복제
- 같은 이미지 캡션을 TABLE과 IMAGE_CAPTION 청크로 이중 생성

콘텐츠 해시는 자동 삭제 기준이 아니라 중복 진단 지표로 사용한다. 공시 원문에 실제로 반복된 문장은 임의로 제거하지 않는다.

14. 검색 시 문맥 확장 규칙

저장 청크는 검색 단위이고 최종 LLM 입력 단위와 같지 않다.

검색 결과를 얻은 뒤 다음 범위에서 문맥을 확장할 수 있다.

- 같은 문서
- 같은 Section
- 같은 청크 유형 또는 원문상 인접한 유형
- 선택된 청크의 직전·직후 청크
- 전체 입력 글자·토큰 예산 안의 범위

서로 다른 Section의 청크는 단순히 순서가 가깝다는 이유로 자동 결합하지 않는다.

TABLE 적중 시에는 같은 표에서 바로 앞뒤 행 조각을 우선하고, TEXT 적중 시에는 같은 HEADING 문맥의 앞뒤 TEXT를 우선한다.

15. 생성 알고리즘

15.1 문서 단위 흐름

COMPLETED DART XML 문서 조회

- Section을 원문 순서로 조회
- Section 전체 경로 생성
- ContentBlock을 원문 순서로 조회
- preamble 처리
- Section별 블록 순회
- TEXT·TABLE·IMAGE_CAPTION 후보 생성
- 작은 후보의 조건부 결합
- 크기·출처·순서 검증
- 문서 단위 저장 결과 교체

15.2 블록 순회 흐름

HEADING

- 누적 TEXT 본문이 있으면 종료
- headingContext 갱신

PARAGRAPH

- 같은 문맥의 TEXT에 추가 시도
- 크기 초과 시 현재 TEXT 종료 후 새 후보
- 긴 단일 문단이면 문장 단위 분리

TABLE

- 누적 TEXT 종료
- 논리적 그리드 생성
- 행 직렬화
- 행 묶음 TABLE 조각 생성
- 중첩 표 재귀 처리
- 조건을 만족하는 인접 작은 TABLE 조각 결합

IMAGE

- 의미 있는 캡션이면 IMAGE_CAPTION 후보

PAGE_BREAK

- 누적 TEXT가 충분한 경우에만 종료

Section 종료

- 남은 TEXT와 후보 확정

16. 원문 추적과 무결성 규칙

모든 저장 청크는 다음 조건을 만족해야 한다.

1. 하나의 `DisclosureDocument`에만 속한다.
2. preamble를 제외하면 하나의 Section에만 속한다.
3. `sourceBlockIds`가 하나 이상 존재한다.
4. 참조한 Block이 같은 문서에 속한다.
5. 참조한 Block의 순서가 오름차순이다.
6. 시작 원문 행이 종료 원문 행보다 뒤에 있지 않다.
7. `chunkSequenceNo`가 문서 안에서 1부터 연속된다.
8. `bodyText`와 `searchText`가 비어 있지 않다.
9. `chunkType`과 원본 Block 유형이 허용된 조합이다.
10. TABLE 출처의 행·셀 인덱스가 원본 JSON 범위 안에 있다.
11. 생성기 이름과 버전이 존재한다.

Fact 또는 답변 근거에서 청크를 사용할 때도 최종적으로 원본 Block과 원문 위치를 반환할 수 있어야 한다.

17. 버전과 재생성 규칙

초기 생성기 정보:

```
generatorName = DartXmlDisclosureChunkGenerator
generatorVersion = dart-xml-chunk-v1
```

다음 변경은 생성 버전을 올려야 한다.

- TEXT 목표·최대 크기 변경
- 문장 분리 방식 변경
- TABLE 머리글 추론 변경
- rowspan·colspan 직렬화 변경
- 중첩 표 중복 제거 방식 변경
- 검색 텍스트 정규화 변경
- 인접 작은 표 결합 규칙 변경

재생성은 문서 단위 전체 교체 방식으로 처리한다.

```
기존 문서 청크 삭제
→ 새 청크 생성·저장
→ 성공 시 생성 완료 상태 갱신
→ 실패 시 전체 롤백
```

파서 버전과 청크 생성 버전을 별도로 기록한다. 파서 결과가 바뀌지 않아도 청크 규칙만 변경해 재생성할 수 있어야 한다.

18. 소규모 검증 대상

전체 3,147개 문서를 바로 생성하지 않고 다음 대표 문서부터 검증한다.

목적	접수번호	기업·공시	확인 항목
소형 수시공시	20230823000334	두산로보틱스 유상증자결정	레이블-값 표, rowspan
최대 크기 문서	20260324000835	KB금융 사업보고서	대용량, 7,544개 블록, 중첩 표
중첩 표 최대	20260515002520	우리기술 분기보고서	최상위 표 392개, 중첩 표 399개
이미지 최대	20240329003141	현대건설 사업보고서	IMAGE_CAPTION 102개
SECTION-4	20250314001069	고려아연 대량보유상황보고서	4단계 Section 경로

이후 공시 그룹과 파일 크기를 섞은 50개 문서로 확대한다.

19. 검증 지표

19.1 필수 무결성 지표

지표	완료 기준
빈 청크	0
존재하지 않는 원본 Block 참조	0
다른 문서의 Block 참조	0
Section 경계 침범	0
원문 행 역전	0
문서 내 청크 순서 중복·누락	0
TABLE 행·셀 범위 오류	0
생성 실패 문서	원인 확인 없이 전체 확대 금지

19.2 분포 지표

다음 값은 고정 합격선보다 분포와 이상치를 확인한다.

- 문서당 전체 청크 수
- 문서당 TEXT·TABLE·IMAGE_CAPTION 수
- **bodyText** 길이의 최소·평균·p50·p90·p95·p99·최대
- 목표 하한 미만 청크 비율
- 일반 최대 초과 청크 수
- 절대 최대 초과 청크 수
- 청크당 원본 Block 수
- TABLE 청크당 표·행·셀 수
- 중첩 표 생성 수와 깊이
- 반복된 머리글이 차지하는 글자 비율
- 동일 문서 안의 완전 동일 청크 비율

19.3 사람이 확인할 항목

- Section 경로만 읽어도 문맥을 이해할 수 있는가?
- HEADING과 본문이 올바르게 연결됐는가?
- 문장이 중간에서 잘리지 않았는가?
- 표의 레이블과 값이 함께 보이는가?
- 분할 표마다 머리글과 단위가 있는가?
- rowspan으로 생략된 상위 레이블이 검색 텍스트에 보완됐는가?
- 중첩 표가 부모와 중복되지 않았는가?
- 청크에서 원본 Block과 행으로 돌아갈 수 있는가?

19.4 검색 품질 검증

대표 질문의 정답 근거 청크가 검색 상위 K개 안에 나오는지 확인한다.

예시:

두산로보틱스 유상증자의 발행 주식 수는?
KB금융 사업보고서에서 연구개발 관련 내용은?
고려아연 대량보유자의 취득자금 원천은?
현대건설 사업보고서에서 정정된 이미지 설명은?

초기 목표는 정답 근거가 **top 5** 또는 **top 10** 안에 포함되는지 측정하는 것이다. 청크 크기는 이 결과를 보고 조정한다.

20. 구현 컴포넌트 권장 구조

```
disclosure/infrastructure/chunking
├─ GeneratedDisclosureChunk
├─ DisclosureChunkType
├─ DisclosureChunkingPolicy
├─ SectionPathResolver
├─ ChunkTextNormalizer
├─ SentenceBoundarySplitter
├─ TextChunkGenerator
├─ TableLogicalGridBuilder
├─ TableTextSerializer
├─ TableChunkGenerator
├─ ImageCaptionChunkGenerator
└─ DartXmlDisclosureChunkGenerator
```

역할:

컴포넌트	책임
DisclosureChunkingPolicy	크기와 결합 관련 v1 설정
SectionPathResolver	부모 Section을 따라 전체 경로 생성
ChunkTextNormalizer	검색 텍스트 공백·줄바꿈 정규화
SentenceBoundarySplitter	긴 문단의 안전한 문장 분리

컴포넌트	책임
<code>TextChunkGenerator</code>	HEADING 문맥과 PARAGRAPH 결합·분리
<code>TableLogicalGridBuilder</code>	rowspan·colspan을 고려한 논리적 표 구성
<code>TableTextSerializer</code>	표·행·셀을 검색 텍스트로 직렬화
<code>TableChunkGenerator</code>	행 묶음, 머리글 반복, 중첩 표 재귀 처리
<code>ImageCaptionChunkGenerator</code>	의미 있는 이미지 캡션 후보 생성
<code>DartXmlDisclosureChunkGenerator</code>	문서 전체 흐름과 최종 순서 통합

각 하위 컴포넌트는 DB 저장과 분리된 순수 로직으로 먼저 구현한다. 그래야 대표 문서 결과를 저장 전에 검사하고 단위 테스트하기 쉽다.

21. 구현 순서

1. `DisclosureChunkingPolicy` 작성
2. `GeneratedDisclosureChunk`와 출처 모델 작성
3. `SectionPathResolver` 구현·테스트
4. `ChunkTextNormalizer` 구현·테스트
5. `SentenceBoundarySplitter` 구현·테스트
6. `TextChunkGenerator` 구현·테스트
7. `TableLogicalGridBuilder` 구현·테스트
8. `TableTextSerializer` 구현·테스트
9. `TableChunkGenerator`와 중첩 표 처리 구현·테스트
10. `ImageCaptionChunkGenerator` 구현·테스트
11. `DartXmlDisclosureChunkGenerator` 통합
12. 대표 문서 5개 결과를 CSV 또는 JSONL로 출력
13. 길이·개수·중복·원문 연결·가독성 분석
14. 50개 소규모 배치 검증
15. `disclosure_chunks` Flyway·Entity·Repository 확정
16. 문서 단위 교체 저장 서비스 구현
17. 전체 배치 생성
18. PostgreSQL 텍스트 검색과 대표 질문 검색 평가

22. 완료 기준

다음 조건을 모두 만족해야 `dart-xml-chunk-v1` 규칙 구현이 완료된 것으로 본다.

- ☐ Section 경로 생성 규칙이 테스트된다.
- ☐ preamble이 누락되지 않는다.
- ☐ HEADING이 다음 PARAGRAPH의 문맥으로 연결된다.
- ☐ 짧은 문단이 같은 문맥 안에서 결합된다.
- ☐ 긴 문단이 의미 경계로 분리된다.
- ☐ TABLE이 논리적 행 텍스트로 변환된다.
- ☐ rowspan·colspan 문맥이 검색 텍스트에 보존된다.
- ☐ 분할 TABLE 청크에 머리글과 단위가 반복된다.

- ☐ 중첩 표가 재귀 처리되고 부모와 중복되지 않는다.
- ☐ 의미 없는 이미지와 PAGE_BREAK 청크가 생성되지 않는다.
- ☐ 모든 청크가 원본 Block과 원문 행으로 역추적된다.
- ☐ 대표 문서 5개의 생성 결과를 사람이 검수한다.
- ☐ 50개 소규모 배치가 무결성 오류 없이 완료된다.
- ☐ 청크 길이와 개수 분포를 분석한다.
- ☐ 대표 질문의 정답 근거가 검색 상위 K개 안에 포함된다.
- ☐ 규칙 변경 시 생성 버전이 함께 변경된다.

23. 확정 규칙과 조정 가능한 설정

23.1 구현 중 유지해야 하는 확정 규칙

- 문서 경계를 넘지 않는다.
- Section 경계를 기본적으로 넘지 않는다.
- TEXT와 TABLE을 섞지 않는다.
- 구조 경계를 글자 수보다 우선한다.
- HEADING은 본문 문맥으로 사용한다.
- TABLE은 행 단위로 직렬화하고 분할한다.
- rowspan-colspan의 문맥을 검색 텍스트에서 보완한다.
- 중첩 표를 재귀 처리하고 중복 저장하지 않는다.
- PAGE_BREAK는 독립 청크를 만들지 않는다.
- 의미 있는 이미지 캡션만 검색에 사용한다.
- 저장 overlap을 사용하지 않고 검색 시 인접 문맥을 확장한다.
- 모든 청크는 원본 Block과 원문 위치를 보존한다.
- 검색 청크의 숫자를 계산 입력으로 직접 사용하지 않는다.

23.2 검증 결과에 따라 조정 가능한 설정

- TEXT 목표 700~1,000자
- TEXT 일반 최대 1,400자
- TABLE 목표 1,000~1,500자
- TABLE 일반 최대 2,000자
- 긴 문장·긴 셀의 절대 최대
- 인접 작은 표 결합 조건
- 검색 시 앞뒤 청크 확장 개수
- 이미지 파일명 포함 여부
- PostgreSQL 검색 방식과 가중치

조정 가능한 값을 변경할 때는 대표 질문 검색 결과와 길이 분포를 함께 기록하고 `generatorVersion`을 올린다.